PotholeGuard

Smart Pothole Detection & Rider Alert System

Updated build guide: web app (PWA) version. No Android Studio. Antigravity prompts + the manual work only you can do.

READ FIRST | What you are building

A web app you open in **Chrome on your Android phone**. You add it to the home screen and it behaves like an app. It uses the camera, an on-device YOLO model, GPS speed, and warns with voice, vibration and a big visual banner. After the first load it works offline. Cost: Rs. 0 (plus a Rs. 300-800 phone holder). API keys needed: none.

Part	Tool	Why this choice
Code	Plain HTML + JavaScript, no build tools	Simplest possible. Nothing to install on your laptop except Antigravity.
AI model	YOLO nano trained in Colab, exported to ONNX	Runs in the browser with ONNX Runtime Web (WebGPU, or WASM fallback).
Phone features	Camera, GPS, orientation, speech, vibration APIs	All built into Android Chrome.
Hosting	Vercel (free)	Gives an HTTPS link. The camera only works on HTTPS.

READ FIRST | How to use this guide

- **MANUAL** = you do it (installs, data, training, measuring, testing).
- **AGENT PROMPT** = paste into Google Antigravity; the agent writes the code.
- Follow the steps in order. For each prompt: new agent conversation, paste the **Master Context** first, then the prompt. Use Plan mode for the big ones. Test on your phone before moving on.

Safety: never touch or look at the phone while riding. Do all app interaction while stopped, and use a helper or recordings for road tests. Alerts are advisory only. Follow traffic laws and wear a helmet.

Limits of the web version (be honest about them in your report): Android Chrome only (iPhone Safari has no vibration). Screen must stay on and Chrome in the foreground. Expect about 8-15 FPS, lower on old phones. If the web MVP proves the idea, a native app can come later.

Roadmap

#	Type	Step	Result
0	MANUAL	Setup accounts and tools	Ready
1	PROMPT 1	App skeleton: camera, HTTPS-ready, installable, offline shell	Live camera + FPS in browser
2	MANUAL 1	Deploy on Vercel, open on phone	Working link on your phone
3	MANUAL 2	Data, train in Colab, export ONNX	pothole.onnx model
4	PROMPT 2	Run the model on camera frames	Live pothole boxes
5	PROMPT 3	Tracking + rider-path corridor	Only stable, in-path potholes
6	MANUAL 3	Mount phone, measure height and pitch, mark distances	Calibration numbers
7	PROMPT 4	Distance estimation + calibration screen	Distance in metres
8	MANUAL 4	Test 5, 10, 15, 20, 25, 30 m	Error table
9	PROMPT 5	GPS speed, time-to-pothole, risk levels	Risk engine
10	PROMPT 6	Voice, vibration, visual alerts	Complete warning system
11	PROMPT 7	Session log + summary screen	Numbers for your report
12	MANUAL 5	Field tests and retrain loop	Real-world results
13	PROMPT 8 (optional)	Crowdsourced pothole map	Phase 7 of your problem statement
14	MANUAL 6	Report and demo	Submission

Step 0 - Setup (manual)

MANUAL - YOU DO THIS | Install and create these

What	Why	How
Google Antigravity	The agent IDE that writes the code.	https://antigravity.google/ - install and sign in with Google. If the site changed, search 'Google Antigravity download'.
Chrome on your Android phone	Runs the app. Update it to the latest version.	Play Store > Chrome > Update.
Google account + Colab	Free GPU to train the model.	https://colab.research.google.com - later choose Runtime > Change runtime type > T4 GPU.
Roboflow (free)	Find datasets, label your own images.	https://roboflow.com and https://universe.roboflow.com
Vercel (free)	Hosts your app on HTTPS in seconds.	https://vercel.com - sign up once (GitHub or Google login works).
Node.js (for the Vercel CLI)	Lets you deploy from the terminal with one command.	https://nodejs.org - install LTS, then run npm install -g vercel.
Netron (optional)	Look inside the model file.	https://netron.app (runs in the browser, nothing to install).
Bike phone holder	Stable mount.	Any handlebar or mirror mount, Rs. 300-800.

Create an empty folder on your laptop called `potholeguard`. In Antigravity choose File > Open Folder and select it. That folder is your whole project.

Master Context (paste first, in EVERY agent conversation)

```
PROJECT: "PotholeGuard" - a phone-only pothole detection and rider alert web app (PWA) for
two-wheelers. It is a student capstone project, so keep the code simple and readable.

STACK: plain HTML, CSS and JavaScript (ES modules). NO frameworks, NO bundler, NO build step. YOLO
model in ONNX format run with ONNX Runtime Web (WebGPU if available, else WASM). Browser APIs:
getUserMedia (rear camera), Geolocation, DeviceOrientation, Screen Wake Lock, SpeechSynthesis,
Vibration, Service Worker. Target: Chrome on Android, served over HTTPS.

PIPELINE: camera frame -> YOLO pothole detector -> tracker -> rider-path check -> monocular distance
-> GPS speed -> time-to-pothole -> risk engine -> voice + vibration + visual warning.

FILES: index.html, styles.css, app.js, config.js, detector.js, tracker.js, geometry.js, risk.js,
alerts.js, logger.js, sw.js, manifest.json, tests.html, folders /lib (third-party libraries) and
/models.

RULES:
1. Every tunable number (confidence threshold, confirm frames, corridor size, camera height, camera
pitch, distance bands, cooldowns) lives ONLY in config.js. Allow overriding values from a settings
panel saved in localStorage.
2. The core pipeline works offline. Do not load libraries from a CDN at runtime: download them into
/lib (use the terminal, for example curl or npm pack) and reference them locally. The service worker
must cache the app shell, /lib and /models.
3. Bump a CACHE_VERSION constant in sw.js on every change I ask for, so my phone loads the new
version.
4. Put pure logic (tracker, geometry, risk) in separate modules with no DOM access. Put simple tests
in tests.html that I can open in a browser and that show PASS/FAIL.
5. Check current official documentation instead of guessing library versions or APIs.
6. After each task, tell me exactly what to test on my phone.
7. Do not add features I did not ask for in the current prompt.
```

Prompt 1 - App skeleton and live camera

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 1

```
TASK: Create the web app skeleton in the current folder and get the live camera working.

1. index.html with a full-screen <video> (rear camera, playsinline, muted) and a transparent
<canvas> overlay on top. A big START button on first load (browsers require a tap before camera,
speech and vibration).
2. app.js: on START request the rear camera with getUserMedia (facingMode environment, ideal
1280x720), start the video, request a Screen Wake Lock, and show a small overlay with video
resolution and measured frame rate (requestVideoFrameCallback if available, else
requestAnimationFrame).
3. Friendly error messages if the camera is blocked, missing, or the page is not on HTTPS.
4. config.js exporting a Config object with placeholder values for every tunable in rule 1 of the
master context.
5. manifest.json + icons (simple generated placeholder icons) so it installs to the home screen as a
standalone, landscape-friendly app.
6. sw.js: cache the app shell so it loads offline after the first visit.
7. A simple settings panel (gear icon) that lists Config values and lets me edit and save them to
localStorage.
8. README.md: how to run locally (python3 -m http.server works for a laptop check) and how to deploy
on Vercel.

ACCEPTANCE: opening index.html through a local server on the laptop shows the webcam and FPS; the
folder deploys on Vercel as-is (static site, no build step).
```

MANUAL - YOU DO THIS | Manual 1 - Deploy and open on your phone

1. Install Node.js (https://nodejs.org, LTS) if you have not already, then run `npm install -g vercel` once.
2. In a terminal, `cd` into your `potholeguard` folder and run `vercel --prod`. Sign in when prompted (GitHub or Google).
3. Vercel prints an https link. Open it in Chrome on your phone. Allow camera permission.
4. Check: camera view shows and FPS is displayed. Then Chrome menu > Add to Home screen (or Install app).
5. Each time the agent changes code, run `vercel --prod` again from the folder. Then close and reopen the app; the cache version bump loads the new code.

Faster phone testing: push the folder to a GitHub repo and import it once at https://vercel.com (Add New Project). After that, every git push auto-deploys - no need to run the CLI command again.

Manual 2 - Data, training and model export

The agent cannot train the AI model. This is your most important manual step, but it is short: get images, train in Colab, export one file.

MANUAL - YOU DO THIS | 2A - Get pothole images

Start with Route A. It is enough for the MVP.

1. Open https://universe.roboflow.com and search 'pothole'.
2. Pick 1-2 datasets with 1,000+ images and a licence that allows your use.
3. Click Download Dataset > format **YOLOv8** > download zip. Unzip it. If you use two datasets, put their images and labels into one dataset folder (train/val).

Route B (optional, for extra accuracy on Indian roads) - RDD2022 India: project page https://github.com/sekilab/RoadDamageDetector, data on figshare https://figshare.com/articles/dataset/RDD2022_-_The_multi-national_Road_Damage_Dataset_released_through_CRDDC_2022/21431547. Download the India files. Only class D40 is a pothole. Use Prompt T1 below to convert it.

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt T1 (optional) - Convert RDD2022 India to YOLO format

```
(Run in a separate folder called dataset_tools. No Master Context needed.)

TASK: Write a Python 3 script convert_rdd_to_yolo.py.
INPUT: a folder with RDD2022 India in Pascal VOC format (images/ and annotations/xml/); path given as
a command line argument.
DO:
1. Keep ONLY boxes of class D40 and map them to class id 0 (pothole). Ignore other classes.
2. Write YOLO txt labels (class x_center y_center w h, normalized). Keep images with no D40 as
negatives (empty txt) but cap negatives at 20%.
3. Random split (seed 42) 80% train / 20% val into dataset/images/{train,val} and
dataset/labels/{train,val}.
4. Write dataset/data.yaml (path, train, val, names {0: pothole}).
5. Optional --merge <other_yolo_dataset_folder> to copy another YOLO dataset in, remapping every
class to 0.
6. Print counts per split and number of pothole boxes.
```

MANUAL - YOU DO THIS | 2B - Add a few of your own Bengaluru images (this makes it original)

- Record short videos on the mounted phone (helper operates it, or go slowly on a quiet road): dry, wet, water-filled potholes, shadows, evening.
- Pull out about 2 frames per second, for example with VLC or `ffmpeg -i ride.mp4 -vf fps=2 frames/%04d.jpg`.
- Upload to a Roboflow project, draw boxes around potholes with one class named **pothole**. (Alternative: https://www.cvat.ai.)
- Start with 200-300 images including some **hard negatives** (manholes, speed breakers, patches, puddles with no pothole; leave them without boxes). Add more after your field tests.
- Export in YOLOv8 format and merge with your base dataset.

MANUAL - YOU DO THIS | 2C - Train in Google Colab (copy these cells)

Zip your final `dataset` folder (it must contain data.yaml) and upload it to Google Drive. Docs: https://docs.ultralytics.com.

```
# Cell 1
!pip install -q ultralytics

# Cell 2 - mount Drive, unzip
from google.colab import drive
drive.mount('/content/drive')
!unzip -q /content/drive/MyDrive/dataset.zip -d /content/
# data.yaml should say: path: /content/dataset train: images/train val: images/val names: {0:
pothole}

# Cell 3 - train (nano = smallest and fastest)
from ultralytics import YOLO
model = YOLO("yolo11n.pt") # or "yolov8n.pt" if not found
model.train(data="/content/dataset/data.yaml", epochs=100, imgsz=640,
            batch=16, patience=20, project="/content/drive/MyDrive/runs", name="pothole_v1")

# Cell 4 - check quality
metrics = model.val()
print("mAP50", metrics.box.map50, "precision", metrics.box.mp, "recall", metrics.box.mr)

# Cell 5 - export to ONNX for the browser
best = YOLO("/content/drive/MyDrive/runs/pothole_v1/weights/best.pt")
best.export(format="onnx", imgsz=640, simplify=True)
# creates best.onnx next to best.pt -> download it
# If the phone is too slow later, re-export with imgsz=416 (or 320)
```

ONNX export guide: https://docs.ultralytics.com/integrations/onnx/.

MANUAL - YOU DO THIS | 2D - Put the model into the app

1. Open `best.onnx` in https://netron.app. Write down the input shape (typically 1 x 3 x 640 x 640) and output shape (for a one-class model typically 1 x 5 x 8400). Check yours; do not assume.
2. Rename it `pothole.onnx` and put it in the `models` folder of your project.
3. Quick sanity check in Colab: `best('photo.jpg', save=True)` to confirm it actually finds potholes.
4. Save your training results (mAP, precision, recall, sample predictions) for the report.

Prompt 2 - Run the model on camera frames

Fill in the two [BRACKETS] from Netron first.

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 2

```
TASK: Run the YOLO pothole model in the browser and draw live detections.

MODEL FACTS (from Netron):
- File: models/pothole.onnx
- Input: [INPUT SHAPE, e.g. 1x3x640x640 float32, RGB, values 0-1, NCHW]
- Output: [OUTPUT SHAPE, e.g. 1x5x8400 = cx, cy, w, h, score per candidate; log a sample at runtime
to verify units]
- One class: pothole

DO:
1. Download onnxruntime-web (official npm package, check current docs at
https://onnxruntime.ai/docs/tutorials/web/ ) into /lib, including the .wasm files, and load it
locally.
2. detector.js: load the model; try the WebGPU execution provider first and fall back to WASM (show
which backend is active on screen). Add a Config option to force WASM.
3. Preprocess each video frame: draw to an offscreen canvas with letterbox resize to the model size,
convert to a Float32Array in NCHW RGB 0-1.
4. Decode the output (it is channels-first: transpose if needed), filter by
Config.confidenceThreshold, apply NMS (IoU 0.45), return boxes in the ORIGINAL video coordinates.
5. Run in a loop that skips frames while an inference is still running (real time beats
completeness).
6. Draw boxes and confidence on the overlay canvas, aligned correctly with the video in any screen
orientation and size. Show inference ms, FPS, and detection count.
7. Update sw.js to cache /lib and models/pothole.onnx so the app works offline after first load. Bump
CACHE_VERSION.
8. Add a "Test image" button that runs the detector on a photo I pick from the phone gallery (easier
debugging).
9. If multi-threaded WASM needs cross-origin isolation, add a vercel.json with the required
COOP/COEP response headers; if that causes problems, use single-threaded WASM and tell me.

ACCEPTANCE: pointing the phone at a pothole photo on a monitor draws a correct box; report the FPS
and backend you measured.
DO NOT: add tracking, distance or alerts yet.
```

If boxes are wrong or missing, tell the agent what you see. Usual causes: output not transposed, 0-255 vs 0-1 scale, RGB vs BGR, letterbox math, overlay scaling. If FPS is low, try the smaller model (imgsz 416 or 320) and force WASM vs WebGPU to compare.

Prompt 3 - Tracking and rider-path corridor

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 3

```
TASK: Make detections stable and decide whether a pothole is in the rider's path.

1. tracker.js (pure logic): lightweight IoU + centroid tracker with simple smoothing of each box.
Each track has id, smoothed box, confidence, hits, misses.
2. A track is CONFIRMED when seen in at least Config.confirmFrames of the last Config.windowFrames
frames (defaults 3 of 5). Delete a track after Config.maxMissedFrames misses. Only confirmed tracks
continue down the pipeline.
3. geometry.js: a trapezoid path corridor from the horizon row (Config.horizonY, fraction of frame
height) to the bottom of the frame, with bottom width, top width and center as fractions of frame
width. For each confirmed track use the bottom-center of its box: return inPath (true/false) and
lateral offset. Add small hysteresis so edge cases do not flicker.
4. Define the corridor behind a function getCorridor() so a curved path from the gyroscope can
replace it later. Implement only the straight one now.
5. Overlay: tentative tracks gray, confirmed in-path tracks red, confirmed off-path tracks yellow;
draw the corridor translucent.
6. Settings panel sliders for horizonY, corridor widths and center, saved in localStorage.
```

```
7. tests.html: a one-frame false detection never becomes confirmed; a track survives one missed
frame; two potholes keep separate ids; a box in the middle is inPath and one at the edge is not.

ACCEPTANCE: tests pass; flickering detections do not turn red; sliders move the corridor live.
```

Manual 3 - Mount the phone and measure

MANUAL - YOU DO THIS | Manual 3 - Calibration measurements

Find a flat, straight, safe, empty area (parking lot or quiet road).

1. Mount the phone firmly, choose portrait or landscape and keep it the same for all later tests.
2. Bike upright, you seated normally. Measure the height of the camera lens above the ground: **H** in metres.
3. Mark distances on the ground at 5, 10, 15, 20, 25, 30 m ahead of the point directly below the camera (tape measure or measuring wheel, chalk marks).
4. Make a test target: a dark, flat 40 cm object (plywood, black mat, or a printed pothole photo on cardboard). It is safer and more repeatable than a real pothole.
5. Write down the phone model, orientation and H. Put H into the settings panel.

Prompt 4 - Distance estimation

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 4

```
TASK: Estimate ground distance to each confirmed pothole with one camera (flat-ground pinhole
model), and add a simple calibration screen.

MATH (geometry.js):
- H = camera height (Config.cameraHeightM), theta = camera pitch below horizontal (radians), fy =
vertical focal length in pixels, cy = image center row in pixels of the video frame
(video.videoHeight / 2).
- For the pixel row y_b of the BOTTOM edge of the pothole box (ground contact):
    alpha = atan((y_b - cy) / fy)
    distance d = H / tan(theta + alpha) (only valid when theta + alpha > 0)
- Lateral offset X = (u - cx) * d / fx for box center column u.

DO:
1. Web browsers do not expose lens focal length. Start from an assumed vertical field of view
(Config.vFovDeg default 50) to get fy = (videoHeight / 2) / tan(vFov / 2), and let me override fy
directly.
2. Pitch: read the DeviceOrientation sensor (beta) when available, low-pass filtered, converted to
camera pitch for my phone orientation; allow a manual pitch override and a trim offset. Show live
pitch on the debug screen.
3. CALIBRATION SCREEN: I place the target at two known distances (default 10 m and 20 m). At each I
tap CAPTURE while the target is detected; the app stores the bottom-edge row y_b. Solve for the best
theta and fy (small grid or Newton search over plausible ranges) so that both distances match, then
save them to settings. Show the resulting error.
4. Median-filter the distance per track over the last 5 frames. Also apply an optional linear
correction d_final = a * d + b from Config (default a=1, b=0).
5. Return null when theta + alpha <= 0 or d > Config.maxDistanceM.
6. Debug screen: raw distance, corrected distance, pitch, H, fy. A "true distance (m)" input and a
"Log sample" button that adds a row (time, track id, raw d, corrected d, true d) to a list; a button
downloads the list as CSV.
7. tests.html: hand-calculated examples (e.g. H = 1.1 m, theta = 10 degrees) pass.

ACCEPTANCE: tests pass; after calibration, a target placed at 15 m reads close to 15 m.
```

Manual 4 - Distance validation (your key experiment)

MANUAL - YOU DO THIS | Manual 4 - Test 5 to 30 m

1. Phone mounted, bike stationary on the calibration spot. Run the calibration screen with the target at 10 m and 20 m.

2. Put the target at 5, 10, 15, 20, 25, 30 m. At each distance log 5 readings with the true distance typed in. Note whether the target was detected at all.

3. Download the CSV and in a spreadsheet compute the mean estimate, absolute error and percentage error per distance.

4. If there is a steady bias, fit estimate vs true with a trendline and set a and b in the settings so corrected matches true.

5. Repeat once on another day or road. Decide the honest usable range, for example 'under 15% error up to 20 m'.

True distance (m)	Detected? (x/5)	Mean estimate (m)	Abs error (m)	Error (%)
5				
10				
15				
20				
25				
30				

Expect and report: far distances are noisy because a 1-2 pixel error in the box edge is many metres far away; pitch changes with braking and suspension; slopes break the flat-ground assumption. 30 m may not be reliable with a phone camera in a browser. That is a valid finding and supports the dedicated-hardware future work.

Prompt 5 - GPS speed, time-to-pothole and risk engine

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 5

```
TASK: Add speed, time-to-pothole (TTP) and the risk engine.

1. speed.js: navigator.geolocation.watchPosition with enableHighAccuracy true, maximumAge 0. Use
coords.speed (m/s) when it is not null; otherwise compute speed from the distance between the last
two fixes (haversine) divided by the time difference. Smooth lightly. If there is no update for 3
seconds mark speed UNKNOWN. Show speed in km/h.
2. risk.js (pure logic). Input per frame: confirmed tracks with inPath, confidence, distance (may be
null) and speed. Output: one state for the most urgent in-path pothole: level NONE / INFO / WARN /
HIGH plus distance, TTP, confidence and speed.
3. TTP = distance / max(speed_mps, Config.minSpeedMps).
4. Levels (all in Config, defaults from my design, to be tuned by tests):
   - 30-20 m: INFO (visual/soft warning)
   - 20-10 m: WARN (vibration + voice)
   - under 10 m: HIGH (stronger warning)
   - Escalate one level if TTP < Config.ttpEscalationSec (default 1.5 s).
   - Alert only when: confirmed AND confidence >= Config.alertConfidence AND inPath AND distance
known AND speed >= Config.minAlertSpeedKmh (default 5).
5. Hysteresis (a level cannot drop instantly) and a per-track cooldown so one pothole does not spam
alerts.
6. Main panel exactly like my design:
   POTHOLE DETECTED / Distance: 24 m / Confidence: 93% / Speed: 38 km/h / Risk: HIGH
7. SIMULATION MODE: sliders for speed and distance so I can test the logic without moving.
8. tests.html: speed 40 km/h (11.1 m/s) with distance 20 m gives TTP about 1.8 s and level WARN; low
confidence gives NONE; off-path gives NONE.

ACCEPTANCE: tests pass; simulation gives the right levels; live speed changes when I walk or drive
with the phone.
```

MANUAL - YOU DO THIS | Manual 5a - Check GPS

- Test outdoors with a clear sky. Browser GPS updates about once per second, so speed reacts slowly. Note this as a limitation.
- Compare the app's speed with the bike speedometer on a supervised slow drive.

Prompt 6 - Alerts (voice, vibration, visual)

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 6

```
TASK: Turn the risk state into rider alerts.

1. alerts.js reacts to risk-state changes.
   - Voice: window.speechSynthesis. WARN: "Pothole ahead. Slow down." HIGH: "Pothole! Slow down
now!". Pick an English (India) voice if available, else any English voice. At most one spoken alert
per Config.alertCooldownSec per pothole. Speech must be unlocked by the START tap (speak a silent
utterance on START).
   - Vibration: navigator.vibrate patterns. WARN = two short pulses. HIGH = one long strong pattern.
Detect support and show a small "vibration not supported" note instead of failing.
   - Fallback beeps using the Web Audio API (different pitch per level) in case speech is not
available.
   - Visual: full-width banner, INFO yellow, WARN orange, HIGH red, large text, readable in
sunlight.
2. Settings toggles: voice, vibration, beeps, volume. Save to localStorage.
3. "Test alerts" buttons (INFO, WARN, HIGH) that work without the camera.
4. Keep the screen awake (re-request the wake lock when the tab becomes visible again).
5. All timing values in Config.

ACCEPTANCE: each level triggers the right banner, voice and vibration on my phone; no alert spam.
```

MANUAL - YOU DO THIS | Manual 5b - Feel and hear it

- Test on the mounted phone with the engine running. Handlebar vibration may hide the phone's vibration; voice may be hard to hear in traffic. Try Bluetooth earphones or a helmet headset.
- Adjust phrases, cooldown and volume until they are clear but not annoying.

Prompt 7 - Session log and summary

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 7

```
TASK: Add logging so I can produce real numbers for my capstone report, without any extra tools.

1. logger.js: START/STOP session buttons. Log every 200 ms: timestamp, FPS, inference ms, track id,
confidence, inPath, raw distance, corrected distance, speed, TTP, risk level, alert fired (Y/N).
Keep in memory and also save to IndexedDB so a page refresh does not lose it.
2. Big "Mark event" buttons: GOOD ALERT, FALSE ALARM, MISSED POTHOLE (for use when stopped or by a
helper). Store them with a timestamp.
3. Record battery level at start and end (Battery Status API if available, otherwise ask me to type
it) and the session length.
4. "Download CSV" button (Blob download).
5. A SUMMARY screen computed in the page: average FPS, average and 95th percentile inference time,
number of alerts, false alarms, missed potholes, good alerts, average warning lead time (seconds
from first alert to the pothole reaching about 2 m distance), battery drop, distance traveled from
GPS, and false alarms per km.

ACCEPTANCE: a 10-minute session produces a CSV and a correct summary screen.
```

Manual 5 - Field tests and improving the model

MANUAL - YOU DO THIS | Manual 5 - Real-world testing on Bengaluru roads

Safety first: a helper operates the phone or you review recorded footage afterwards. Never touch the phone while riding. Ride at safe speeds.

Run at least one session per condition and fill this in from the summary screen:

Condition	Sessions	Detection rate	False alarms	Missed	Avg FPS	Battery drop
Daylight						
Cloudy						
Rain						
Wet road						
Water-filled potholes						
Evening						
Night						

Improvement loop (2-3 rounds):

1. Find the frames where the app gave false alarms or missed potholes (use screen recordings or saved frames).
2. Add them to Roboflow, label them (false alarms become hard negatives), add to the dataset.
3. Retrain in Colab, export a new ONNX, replace models/pothole.onnx, bump the cache version, redeploy with vercel --prod (or a git push if connected).
4. Re-test the same route and compare before and after. This comparison is strong evidence for your report.

Wet-weather tests: use a waterproof pouch with a clear camera window and keep the phone out of direct sun for long periods to avoid overheating.

Prompt 8 (optional) - Crowdsourced pothole map

Only after the MVP works. This is Phase 7 of your problem statement.

MANUAL - YOU DO THIS | Before Prompt 8

1. Go to https://console.firebase.google.com > Add project (free Spark plan) > Add a Web app > copy the firebaseConfig snippet.
2. Enable Authentication > Anonymous and Firestore Database (test mode while developing; add rules before sharing).
3. Paste the firebaseConfig into the prompt where marked. The map uses OpenStreetMap through Leaflet, so no map key is needed.

AGENT PROMPT - PASTE IN ANTIGRAVITY | Prompt 8 (optional)

```
TASK: Add OPTIONAL crowdsourced pothole reporting and a map. OFF by default; it must never slow down
or block real-time warnings.

FIREBASE CONFIG: [PASTE firebaseConfig HERE]

1. Use the Firebase web SDK (modular; check current docs) with Anonymous Auth and Firestore. This
feature may load its scripts from a CDN, since it needs internet anyway.
2. When a WARN or HIGH alert fires and I have turned on "Share anonymized detections", upload one
document: latitude and longitude rounded to about 5 decimals, timestamp, confidence, severity (small
/ large, from box size and known distance), and a geohash. NO images and NO personal data.
3. Queue reports offline and send them when the connection returns.
4. A location counts as VERIFIED when 3 different anonymous users report within 15 m in 14 days. A
simple client-side query and grouping by geohash is fine.
5. Map screen with Leaflet + OpenStreetMap tiles: verified potholes red, unverified orange, and my
position.
6. A privacy panel that states exactly what is shared, with a switch to stop sharing and a button to
delete my uploads.

ACCEPTANCE: two test devices (or two simulated users) reporting the same spot produce one verified
marker.
```

Manual 6 - Capstone report and demo

MANUAL - YOU DO THIS | Manual 6 - Package your results

1. Problem and motivation: rain, hidden potholes, two-wheeler accidents in India.
2. System design: the pipeline from your problem statement, and why phone-only and browser-based (zero install, zero hardware cost).
3. Dataset: sources (Roboflow, RDD2022 India, your Bengaluru images), counts, labelling, splits.
4. Model: YOLO nano, training setup, precision, recall, mAP, before and after retraining.
5. Browser deployment: ONNX Runtime Web, WebGPU vs WASM, FPS, inference time, battery, heat.
6. Distance method: formulas, calibration, and the error table from Manual 4.
7. Alert logic: thresholds (configurable), results by condition from Manual 5.
8. Limitations (be honest) and future work: native app, dedicated hardware, night vision, waterproof enclosure, crowdsourced map, speed breakers and manholes, authority dashboard.

Demo checklist: 2-minute screen recording of detection and warnings; a clip from a real ride; the distance error chart; the before/after retraining numbers; the live Vercel link and the code folder or repo.

Claim to prove: 'A normal smartphone can detect a pothole early enough to warn a two-wheeler rider.' Back it with detection rate, usable distance range and warning lead time in seconds.

Appendix A - Links

Purpose	Link
Google Antigravity	https://antigravity.google/
Vercel (hosting)	https://vercel.com
Node.js (for Vercel CLI)	https://nodejs.org
Google Colab	https://colab.research.google.com
Ultralytics docs	https://docs.ultralytics.com
Ultralytics ONNX export	https://docs.ultralytics.com/integrations/onnx/
ONNX Runtime Web	https://onnxruntime.ai/docs/tutorials/web/
Roboflow / Universe datasets	https://roboflow.com , https://universe.roboflow.com
RDD2022 project	https://github.com/sekilab/RoadDamageDetector
RDD2022 data (figshare)	https://figshare.com/articles/dataset/RDD2022_-_The_multi-national_Road_Damage_Dataset_released_through_CRDDC_2022/21431547
CVAT labelling (optional)	https://www.cvat.ai
Netron model viewer	https://netron.app
Firebase console (optional)	https://console.firebase.google.com

Appendix B - Troubleshooting

- **Camera does not start:** the page must be on HTTPS (Vercel link) and camera permission allowed for the site in Chrome. Local file:// pages will not work.
- **Old version keeps loading:** the service worker cached it. Ask the agent to bump CACHE_VERSION, redeploy, then close the app fully and reopen. Or Chrome > Settings > Site settings > clear data for the site.

- **No vibration:** works only on Android Chrome, after a tap on the page, with the tab visible. Some phones disable it in battery-saver mode.
- **No voice:** unlocked only after the START tap. Check media volume and that an English voice is installed (Android Settings > Text-to-speech).
- **Low FPS:** try WebGPU vs WASM, a 416 or 320 model, a lower camera resolution (640x480), and close other apps. Let the phone cool down.
- **Boxes offset or wrong size:** letterbox or overlay scaling issue. Ask the agent to add a test image with known box positions.
- **Distance far off:** re-run the calibration screen, check H, phone orientation, and that the camera has not been re-tilted.
- **Agent invents APIs or versions:** tell it to read the current official docs and name the page it used.
- **Agent changes too much:** keep a copy of the folder before each prompt, then repeat with 'Only modify files X and Y'.

Appendix C - Prompting tips

- One prompt per conversation; test on the phone before the next one.
- Use Plan mode for Prompts 2, 4, 5 and 8 and read the plan before it builds.
- Ask the agent to explain each new file in two lines so you can defend it in your viva.
- Give real numbers (model shapes, camera height, distances) instead of letting it guess.
- Copy the whole project folder to a backup (or use Git) after each working prompt.